MAIE 6000C: Engineering AI Products: From Prototype to Production

Week 6 — Background Jobs, Async Workflows, and Worker-Based Processing

Program: MSc in AI & Entrepreneurship

Term: Fall 2026/27

Instructor: Lefteris NTAFLOS, PhD

The Hong Kong University of Science and Technology

From system memory to system execution

Week 4: What must the system remember?
→ records, state, migrations, database truth

Week 5: How do components communicate?
→ APIs, contracts, validation, errors

Week 6: What work happens after the response?
→ jobs, workers, retries, background execution

Week 7: Can the team demonstrate the complete vertical slice?



The key design question

What work should happen outside the immediate request-response cycle?

Keep inside the request:

- request validation;
- authentication and authorization;
- creating the main record;
- creating a durable job;
- returning an honest initial state.

Move to the worker when work is:

- long-running or unpredictable;
- dependent on an AI or external service;
- failure-prone or retryable;
- batch-oriented or scheduled;
- not required for the client's immediate response.

Examples:

- AI analysis or classification;
- OCR or document extraction;
- generating reports;
- sending batches of notifications;
- scheduled reprocessing;
- calling an unreliable external service.

“Async” Can Mean Different Things

Do not use “async” to mean four different things

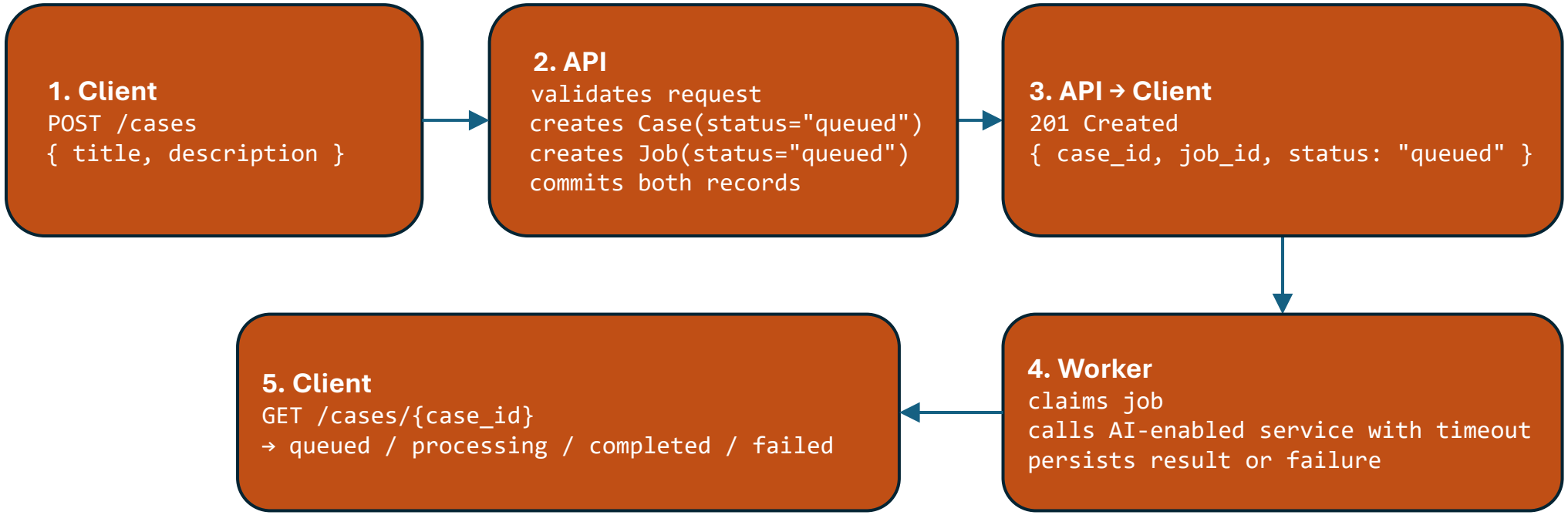
Term	What it means	What it does not guarantee
`async def / await`	Non-blocking I/O within one application process	Durable background work
In-process background task	Work begins after a response	Survival after process crash
Async workflow	Client receives a response before final result exists	Safe worker retry behavior
Persistent job + worker	Durable work processed separately from API	Automatic correctness without design

Key distinction:

- `async def` does not automatically create a reliable worker system.

Reference Workflow: Accept Now, Process Later

A truthful asynchronous workflow





A Job Is a Persistent Contract

The job record makes work durable

A minimal job commonly needs:

- `id`
- `resource_id or case_id`
- `status`
- `attempt_count`
- `available_at or next_attempt_at`
- `claimed_at and/or worker_id`
- `safe failure code or message`
- `created_at, updated_at`

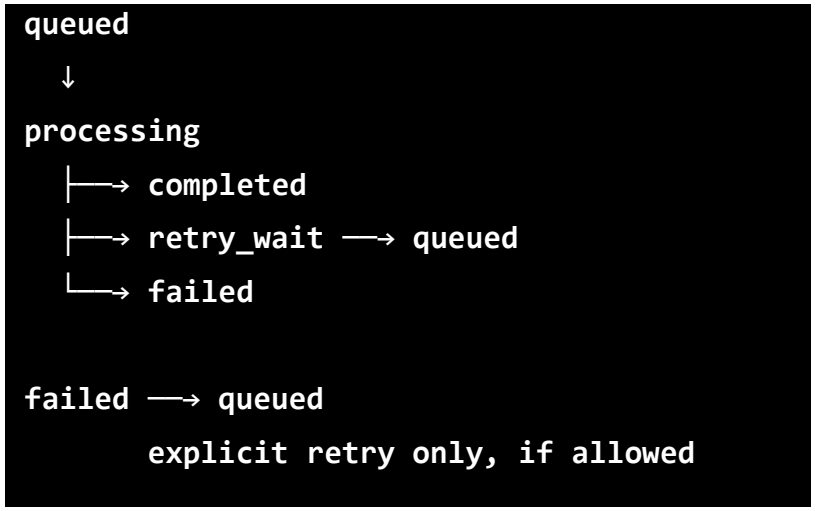
Important design rule

Create the main resource and its initial job together.

- Otherwise, the system may create a record that is never processed.

Design a Small, Truthful Job Lifecycle

Start with a small lifecycle



State	Meaning
queued	Work exists and is waiting
processing	A worker has claimed the work
retry_wait	A temporary failure occurred; retry is scheduled
completed	Result was persisted successfully
failed	No automatic retry remains

Do not add states that no component can explain or test.

Assign Ownership of State Changes

Who may change which state?

Component	Owns
API	validates input; creates resource and job; sets initial state
Worker	claims work; marks processing; persists result, retry, or failure
AI-enabled service	returns an analysis result or an error
Client	submits work and retrieves state
Scheduler, if used	creates future jobs; does not perform all work itself
Database	preserves state and enforces constraints

Boundary rule

The client should not directly set worker-owned processing states.

A Worker Is a Loop, Not a Magic Container

Basic worker logic

```
while worker_is_running:
    job = claim_next_available_job()

    if job is None:
        sleep_briefly()
        continue

    try:
        mark_processing(job)
        result = call_processor_with_timeout(job)
        validate_result(result)
        persist_result_and_mark_completed(job, result)

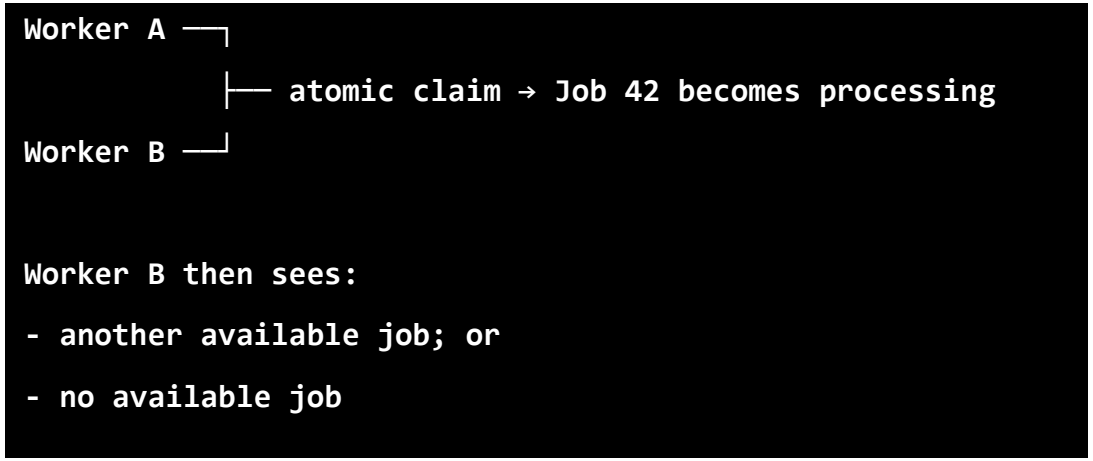
    except RetryableError:
        schedule_bounded_retry(job)

    except Exception:
        persist_safe_failure_and_mark_failed(job)
```

Worker responsibilities

- claim safely;
- use finite timeouts;
- persist every meaningful outcome;
- retry only when appropriate;
- avoid leaving work ambiguous.

Claiming Work Prevents Duplicate Processing



Useful approaches:

- database transaction with a conditional update;
- row locking;
- a claim timestamp or lease;
- worker identifier;
- stale-job recovery after worker crash.

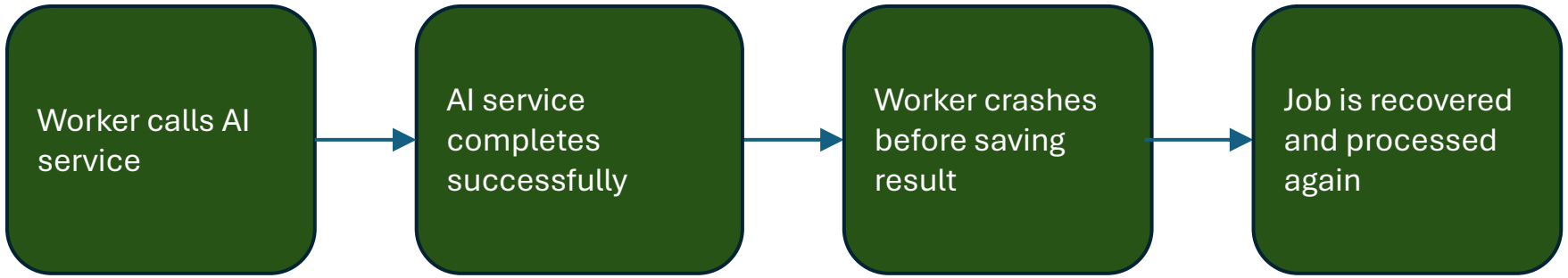
Avoid

```
SELECT queued job
... delay ...
UPDATE job to processing
```

“Exactly Once” Is Usually Not a Safe Assumption

Work may happen more than once

- Example:



Design for at-least-once processing

- make completion writes safe to repeat;
- avoid duplicate external side effects;
- use idempotency keys where duplicate actions matter;
- record enough state to detect or manage replay.

Retry With a Policy, Not With Hope

A retry policy needs:

- maximum attempts;
- delay or backoff;
- next retry time;
- terminal failure behavior;
- visible status for the client.

Not every error should be retried

Failure	Retry?	Typical response
AI timeout	Often yes	bounded retry with delay
Temporary service outage	Often yes	retry later
Rate limit	Often yes	respect delay / backoff
Invalid input	No	mark failed
Malformed AI output	Usually no	reject and record failure
Worker crash	Recoverable	reclaim or recover safely

Failure Must Be Visible Through the API

Existing resource failure is not necessarily HTTP 500

```
{
  "id": "case_123",
  "status": "failed",
  "failure": {
    "code": "AI_TIMEOUT",
    "message": "Analysis could not be completed. Try again later."
  }
}
```

Situation	Appropriate API behavior
Existing case failed during processing	`200 OK + status: "failed"`
Case does not exist	`404 Not Found`
Invalid create request	`422 Unprocessable Content`
API cannot serve request	`5xx response`

Do not expose stack traces, secrets, or raw internal hostnames.

Scheduled and Batch Work Use the Same Pattern

Scheduled and batch workflows still need durable jobs

```
Scheduler → creates durable job → worker processes job
```

```
Batch request → parent record + many jobs → worker pool
```

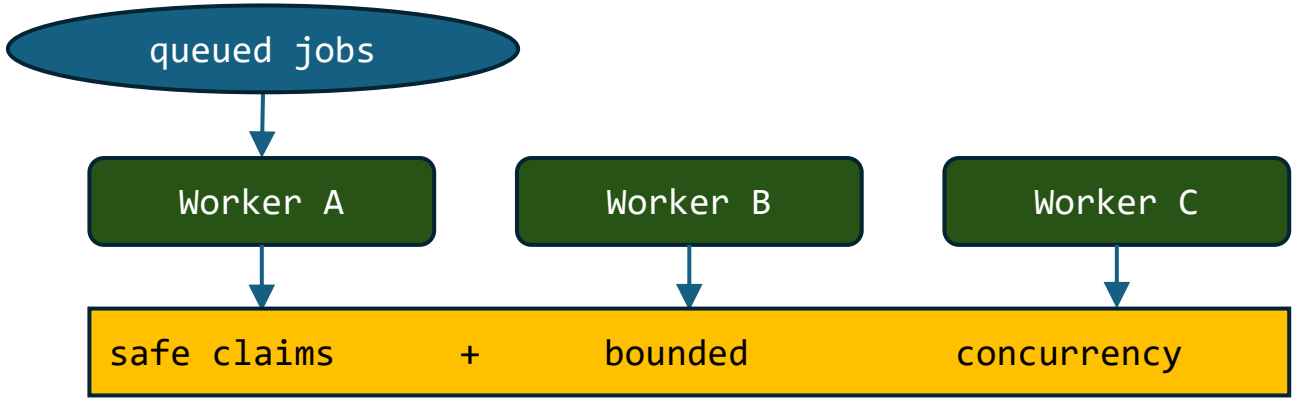
Examples:

- nightly report generation;
- periodic re-analysis;
- processing uploaded files in batches;
- retrying eligible failed work;
- scheduled data refresh.

Rule

A scheduler should normally enqueue work, not become the only place where business processing happens.

Worker-Based Parallelism Is Useful, Not Automatic



Before adding workers, ask:

- Can two workers claim the same job?
- Is the AI service rate-limited?
- Can the database handle the load?
- Are jobs independent?
- Does duplicate work cause harm?
- Is one worker already enough for Week 7?

Minimum Week 7 Worker Vertical Slice

```
POST /<resource>
  → validates input
  → creates resource + durable job
  → returns truthful initial state
```

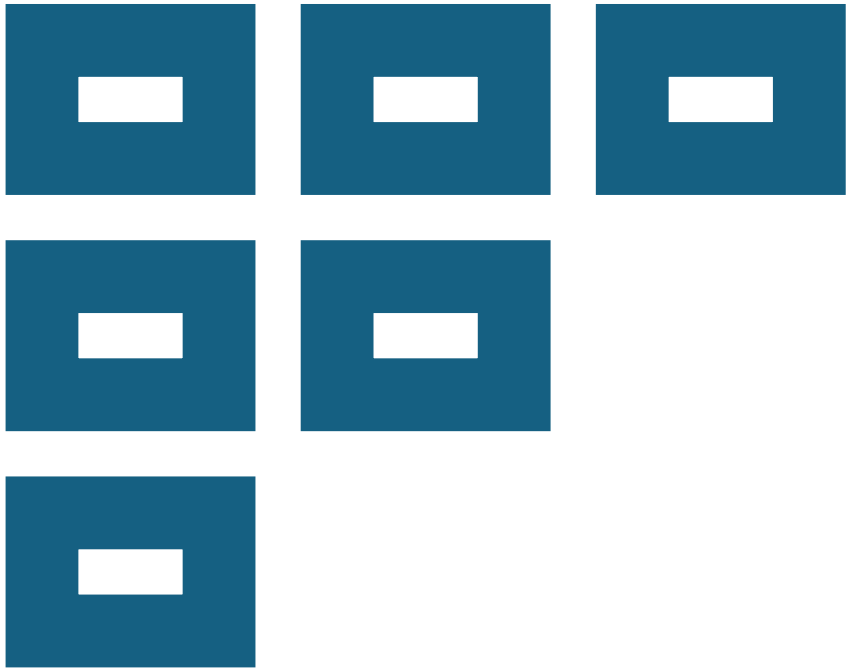
```
Worker
  → claims job
  → processes one meaningful task
  → persists result or failure
```

```
GET /<resource>/{id}
  → returns current status and persisted
result/failure
```

Minimum evidence:

- API create endpoint;
- visible queued/processing/completed/failed state;
- real worker process;
- persisted result or failure;
- one end-to-end test;
- one documented failure decision.

10 - Minute Team Design Check



For your primary workflow, write one answer for each:

- What step belongs outside the API request cycle?
- What resource and job will be persisted?
- What are your four or five meaningful states?
- Which state changes belong to the API?
- Which state changes belong to the worker?
- What failure will the client be able to see?
- What is explicitly out of scope before Week 7?

Output: one small worker-path card per team.

Key Takeaway and Transition to Lab

Reliable background work is visible, durable, and recoverable

durable record
+ durable job
+ clear ownership
+ finite timeout
+ bounded retry
+ persisted result or failure
+ truthful API state
= credible async workflow

Lab: Observe the starter worker path, then design your own Week 7 worker path.

